

React mein TypeScript ka use, Hinglish explanation

TypeScript, JavaScript ke upar ek **type-safety layer** add karta hai. Iska main benefit ye hai ki code run karne se pehle hi editor aur TypeScript compiler bahut saari mistakes detect kar lete hain.

React mein TypeScript use karte waqt aap:

- Components ke props define kar sakte ho
 - State ki possible values control kar sakte ho
 - Events ko accurately type kar sakte ho
 - Context ke data ka structure define kar sakte ho
 - Reducer actions ko restrict kar sakte ho
 - Better autocomplete aur inline documentation paa sakte ho
-

1. React project mein TypeScript install karna

Next.js, Remix, Gatsby aur Expo jaise production-grade React frameworks mein TypeScript support available hai.

Agar existing React project mein TypeScript add karna ho, to React ki type definitions install karo:

```
1 npm install --save-dev @types/react @types/react-dom
```

Ye packages kya karte hain?

- `@types/react`: React components, Hooks, events, children aur React APIs ke TypeScript types provide karta hai.
- `@types/react-dom`: React DOM se related APIs ke types provide karta hai.

tsconfig.json configuration

tsconfig.json mein do important cheezein honi chahiye:

1. lib mein dom included hona chahiye.
2. jsx option kisi valid JSX mode par set hona chahiye.

Most applications ke liye preserve sufficient ho sakta hai:

```
1 {
2   "compilerOptions": {
3     "lib": ["dom", "esnext"],
4     "jsx": "preserve"
```

```
5   }  
6   }
```

Agar lib option diya hi nahi gaya, to dom default roop se included hota hai.

2. .tsx file extension

Jis TypeScript file mein JSX likha ho, us file ka extension .tsx hona chahiye:

```
1  App.tsx  
2  Button.tsx  
3  Form.tsx
```

Normal TypeScript file jisme JSX nahi hai, uske liye .ts use kar sakte ho:

```
1  types.ts  
2  utils.ts  
3  api.ts
```

Simple rule:

- TypeScript + JSX → .tsx
 - Sirf TypeScript → .ts
-

3. Component props ko type karna

JavaScript React aur TypeScript React likhne ka pattern almost same hota hai. Main difference ye hai ki TypeScript mein component ke props ka type define kar sakte ho.

Inline prop type

```
1  function MyButton({ title }: { title: string }) {  
2    return <button>{title}</button>;  
3  }
```

Yahan:

```
1  { title }: { title: string }
```

ka matlab hai:

- Component ko title prop milega
- title ki value string honi chahiye

Correct:

```
1  <MyButton title="Save" />
```

Incorrect:

```
1 <MyButton title={100} />
```

TypeScript doosre example par error dega, kyunki 100 number hai, jabki title ko string hona chahiye.

Props ke liye interface

Jab component mein multiple props ho jayein, inline type messy lag sakta hai. Aise case mein interface use kar sakte ho:

```
1 interface MyButtonProps {
2   /** Button ke andar dikhne wala text */
3   title: string;
4
5   /** Button disabled hai ya nahi */
6   disabled: boolean;
7 }
8
9 function MyButton({ title, disabled }: MyButtonProps) {
10   return (
11     <button disabled={disabled}>
12       {title}
13     </button>
14   );
15 }
```

Usage:

```
1 <MyButton
2   title="I'm a disabled button"
3   disabled={true}
4 />
```

Isse component ka contract clear ho jata hai:

- title required string hai
- disabled required boolean hai

Props describe karne ke liye type ya interface, dono use kiye ja sakte hain.

Union types ka use

Agar prop ya variable ko kuch limited values mein se ek hi value accept karni ho, to union type useful hai:

```
1 type Status = "idle" | "loading" | "success" | "error";
```

Ab Status type ka variable sirf in chaar strings mein se ek ho sakta hai.

```
1 let status: Status = "loading";
```

Ye incorrect hoga:

```
1 let status: Status = "pending";
```

4. Hooks ke saath TypeScript

@types/react mein built-in React Hooks ke types already available hote hain. Isliye Hooks ke liye normally alag setup nahi karna padta.

TypeScript kai baar aapke code ko dekhkar automatically type samajh leta hai. Isse **type inference** kehte hain.

5. useState ko type karna

Automatic inference

```
1 const [enabled, setEnabled] = useState(false);
```

Initial value false dekhkar TypeScript infer kar leta hai ki:

- enabled ek boolean hai
- setEnabled boolean value ya boolean return karne wala function accept karega

Correct:

```
1 setEnabled(true);  
2 setEnabled(false);
```

Incorrect:

```
1 setEnabled("yes");
```

Explicit type

Aap manually bhi type de sakte ho:

```
1 const [enabled, setEnabled] = useState<boolean>(false);
```

Lekin simple values ke liye ye generally necessary nahi hai, kyunki TypeScript already type infer kar leta hai.

Explicit type tab zyada useful hota hai jab state ki multiple possible values hon.

```
1 type Status = "idle" | "loading" | "success" | "error";
2
3 const [status, setStatus] = useState<Status>("idle");
```

Ab ye valid hai:

```
1 setStatus("loading");
2 setStatus("success");
```

Lekin ye invalid hai:

```
1 setStatus("pending");
```

Related state ko object mein rakhna

Request ki state ko alag-alag booleans mein rakhne ke bajay object union ke through represent kiya ja sakta hai:

```
1 type RequestState =
2   | { status: "idle" }
3   | { status: "loading" }
4   | { status: "success"; data: any }
5   | { status: "error"; error: Error };
```

Phir:

```
1 const [requestState, setRequestState] =
2   useState<RequestState>({ status: "idle" });
```

Is structure ka benefit ye hai ki har status ke saath relevant information attach ho sakti hai:

- success ke saath data
- error ke saath error
- idle aur loading ko extra values ki zarurat nahi

Example:

```
1 setRequestState({
2   status: "success",
3   data: responseData
4 });
```

Ya:

```
1 setRequestState({
```

```
2   status: "error",
3   error: new Error("Request failed")
4 });
```

6. useReducer ko type karna

useReducer complex state logic ke liye use hota hai. TypeScript ke saath humein mainly do cheezon ka structure define karna hota hai:

1. State ka structure
2. Possible actions ka structure

State type

```
1 interface State {
2   count: number;
3 }
```

Iska matlab reducer ki state mein count hoga aur wo number hoga.

Action union

```
1 type CounterAction =
2   | { type: "reset" }
3   | { type: "setCount"; value: State["count"] };
```

Reducer sirf do actions accept karega:

```
1 { type: "reset" }
```

Ya:

```
1 { type: "setCount", value: 5 }
```

State["count"] ka matlab hai: State ke count property ka type use karo. Kyunki count number hai, value bhi number hoga.

Initial state

```
1 const initialState: State = {
2   count: 0
3 };
```

Typed reducer

```
1 function stateReducer(
2   state: State,
3   action: CounterAction
```

```

4 ): State {
5   switch (action.type) {
6     case "reset":
7       return initialState;
8
9     case "setCount":
10      return {
11        ...state,
12        count: action.value
13      };
14
15     default:
16      throw new Error("Unknown action");
17   }
18 }

```

Yahan types clearly batate hain:

- state ka type State hai
- action ka type CounterAction hai
- Function State return karega

Use:

```

1 const [state, dispatch] =
2   useReducer(stateReducer, initialState);

```

Dispatch examples:

```

1 dispatch({
2   type: "setCount",
3   value: state.count + 5
4 });
5
6 dispatch({
7   type: "reset"
8 });

```

Agar incorrect action ya wrong value dispatch karoge, TypeScript error show karega.

7. useContext ko type karna

Context ka use component tree mein data share karne ke liye hota hai, bina har level par props pass kiye.

Meaningful default value ke saath

```
1 type Theme = "light" | "dark" | "system";
2
3 const ThemeContext =
4   createContext<Theme>("system");
```

Yahan context ki value sirf teen options mein se ek ho sakti hai:

- "light"
- "dark"
- "system"

Custom Hook:

```
1 const useGetTheme = () => useContext(ThemeContext);
```

State:

```
1 const [theme, setTheme] =
2   useState<Theme>("light");
```

Provider:

```
1 <ThemeContext value={theme}>
2   <MyComponent />
3 </ThemeContext>
```

Child component ke andar:

```
1 const theme = useGetTheme();
```

Jab sensible default value na ho

Kabhi context ke liye meaningful default value available nahi hoti. Tab null use kiya ja sakta hai:

```
1 type ComplexObject = {
2   kind: string;
3 };
4
5 const Context =
6   createContext<ComplexObject | null>(null);
```

Ab context ki value:

- ComplexObject ho sakti hai
- Ya null ho sakti hai

Problem ye hai ki har consumer ko null handle karna padega. Isliye custom Hook ke andar runtime check kar sakte hain:

```
1  const useGetComplexObject = () => {
2    const object = useContext(Context);
3
4    if (!object) {
5      throw new Error(
6        "useGetComplexObject must be used within a Provider"
7      );
8    }
9
10   return object;
11  };
```

Is check ke baad TypeScript samajh leta hai ki returned object null nahi hai.

Iska ek aur benefit hai: agar developer component ko Provider ke bahar use kare, to clear runtime error milega.

8. useMemo

useMemo kisi function se calculate hui value ko memoize karta hai. Dependencies change hone par calculation dobara run hoti hai.

```
1  const visibleTodos = useMemo(
2    () => filterTodos(todos, tab),
3    [todos, tab]
4  );
```

TypeScript callback ke return value se visibleTodos ka type infer kar leta hai. Normally manually type dene ki zarurat nahi hoti.

Provided content ke according, React Compiler values aur functions ko automatically memoize kar sakta hai, jisse manual useMemo ki need kam ho sakti hai.

9. useCallback

useCallback function ka stable reference maintain karta hai, jab tak dependencies same rehti hain.

```
1  const handleClick = useCallback(() => {
```

```
2 // Logic
3 }, [todos]);
```

TypeScript strict mode mein callback parameters ke types dene pad sakte hain.

Example:

```
1 const handleChange =
2   useCallback<
3     React.ChangeEventHandler<HTMLInputElement>
4   >((event) => {
5     setValue(event.currentTarget.value);
6   }, [setValue]);
```

Yahan:

```
1 React.ChangeEventHandler<HTMLInputElement>
```

batata hai ki function HTML input ke change event ko handle karega.

10. DOM events ki typing

Inline event handler mein TypeScript aksar event ka type automatically infer kar leta hai:

```
1 <input
2   onChange={(event) => {
3     setValue(event.currentTarget.value);
4   }}
5 />
```

Lekin jab event handler ko separate function mein extract karte ho, to event type explicitly dena pad sakta hai:

```
1 function handleChange(
2   event: React.ChangeEvent<HTMLInputElement>
3 ) {
4   setValue(event.currentTarget.value);
5 }
```

Yahan:

- `React.ChangeEvent` change event ka React type hai
- `HTMLInputElement` batata hai ki event input element se aa raha hai

Agar kisi event ka specific React type available na mile, to base type use kiya ja sakta hai:

```
1 React.SyntheticEvent
```

`SyntheticEvent` React events ka base type hai.

Practical trick

Agar event ka type yaad na ho, to event ko temporarily inline likho aur editor mein parameter ke upar hover karo:

```
1 <input onChange={(event) => {}} />
```

Editor inferred type show kar dega. Us type ko phir extracted handler mein use kar sakte ho.

11. children ko type karna

React component ke children type karne ke do common approaches hain.

React.ReactNode

```
1 interface ModalRendererProps {  
2   title: string;  
3   children: React.ReactNode;  
4 }
```

React.ReactNode broad type hai. Ye JSX elements ke saath JavaScript primitives jaise strings aur numbers ko bhi represent kar sakta hai.

Example:

```
1 <ModalRenderer title="Information">  
2   Some text  
3 </ModalRenderer>
```

Ya:

```
1 <ModalRenderer title="Information">  
2   <p>Some JSX</p>  
3 </ModalRenderer>
```

React.ReactElement

```
1 interface ModalRendererProps {  
2   title: string;  
3   children: React.ReactElement;  
4 }
```

React.ReactElement specifically JSX element represent karta hai. Ye plain string ya number se zyada restricted hai.

Simple difference:

- `ReactNode`: JSX, text aur doosri renderable values ke liye broad type
- `ReactElement`: JSX element ke liye narrower type

Provided guide ke according TypeScript se reliably ye enforce nahi kiya ja sakta ki component ke children sirf kisi particular JSX element type ke hon, jaise sirf `<li>` elements.

12. Inline styles ko type karna

React ke inline style object ke liye `React.CSSProperties` use kar sakte ho:

```
1 interface MyComponentProps {  
2   style: React.CSSProperties;  
3 }
```

Component:

```
1 function MyComponent({  
2   style  
3 }: MyComponentProps) {  
4   return <div style={style}>Hello</div>;  
5 }
```

Usage:

```
1 <MyComponent  
2   style={{  
3     color: "red",  
4     backgroundColor: "black"  
5   }}  
6 />
```

Isse:

- Valid CSS properties ka autocomplete milta hai
 - Invalid property names identify ho sakte hain
 - Style prop ka structure clear rehta hai
-

Most important takeaways

1. JSX wali TypeScript files mein `.tsx` use karo

```
1 Component.tsx
```

2. Props ko type ya interface se define karo

```
1 interface ButtonProps {  
2   title: string;  
3   disabled: boolean;  
4 }
```

3. Type inference ko unnecessary override mat karo

Ye sufficient hai:

```
1 const [enabled, setEnabled] = useState(false);
```

Har baar ye likhna necessary nahi:

```
1 const [enabled, setEnabled] =  
2   useState<boolean>(false);
```

4. Union types state ko safe banate hain

```
1 type Status =  
2   | "idle"  
3   | "loading"  
4   | "success"  
5   | "error";
```

5. useReducer mein state aur actions dono type karo

```
1 interface State {  
2   count: number;  
3 }  
4  
5 type Action =  
6   | { type: "reset" }  
7   | { type: "setCount"; value: number };
```

6. Nullable context ke liye custom Hook banao

```
1 const value = useContext(Context);  
2  
3 if (!value) {  
4   throw new Error("Provider is missing");  
5 }
```

7. Extracted event handlers mein event type do

```
1 function handleChange(  
2   event: React.ChangeEvent<HTMLInputElement>,  
3   value: string,  
4   onChange: (value: string) => void,  
5   type: string,  
6   checked: boolean,  
7   disabled: boolean,  
8   name: string,  
9   id: string,  
10  ) {  
11   // ...  
12 }
```

```
2   event: React.ChangeEvent<HTMLInputElement>
3 ) {}
```

8. Children ke liye mostly React.ReactNode

```
1 children: React.ReactNode;
```

9. Inline styles ke liye React.CSSProperties

```
1 style: React.CSSProperties;
```

Beginner ke liye recommended learning order

Is page ko effectively samajhne ke liye ye sequence useful rahega:

1. Basic TypeScript types: string, number, boolean
2. Object types
3. type aur interface
4. Union types
5. Type inference
6. Generic syntax, jaise useState<Status>
7. React component props
8. useState
9. DOM events
10. children
11. useContext
12. useReducer
13. useMemo aur useCallback

Starting mein saare advanced React types yaad karne ki zarurat nahi hai. Props, useState, events, children aur context par pehle focus karna practical rahega.

Page mein diye gaye external resources

React ke internal documentation links aur Mozilla/MDN links ko neeche include nahi kiya gaya hai.

Framework TypeScript setup

- [Next.js TypeScript guide](#)
- [Remix TypeScript guide](#)
- <https://www.gatsbyjs.com/docs/how-to/custom-configuration/typescript/>
- [Expo TypeScript guide](#)

TypeScript documentation

- [TypeScript](#)
- [TypeScript JSX configuration](#)
- <https://www.typescriptlang.org/docs/handbook/2/objects.html>
- [TypeScript union types](#)
- <https://www.typescriptlang.org/docs/handbook/2/types-from-types.html>
- <https://www.typescriptlang.org/docs/handbook/type-inference.html>
- <https://www.typescriptlang.org/docs/handbook/>
- <https://devblogs.microsoft.com/typescript/>
- <https://www.typescriptlang.org/play>

React types and examples

- <https://github.com/DefinitelyTyped/DefinitelyTyped/blob/master/types/react/index.d.ts>
- [React DOM event type definitions](#)
- [ReactDOM and ReactDOM Playground example](#)

Community resources

- [React TypeScript Cheatsheet](#)
- <https://discord.com/invite/typescript>
- <https://www.npmjs.com/package/@types/react>
- <https://www.npmjs.com/package/@types/react-dom>

Explanation approach: Page ke original structure ko simplify karke installation, component typing, Hooks aur reusable React types ke order mein arrange kiya gaya hai. Concepts ke saath practical meaning aur error-prevention benefit add kiya gaya hai, bina page ke scope se bahar jaaye.